Lecture 9
GitHub Features: Issues, Actions, Releases, and Protection

Tracking work, automating checks, protecting `main`, publishing versions

Learning goals

- Use Issues and closing keywords to connect a problem, its fix and its pull request.
- Read and write a small GitHub Actions workflow that runs your tests on every push.
- Explain what branch protection does and which rules matter most.
- Create tags and releases, and choose version numbers (semantic versioning).
- Know what GitHub Pages and the `.github` folder offer.

1 What GitHub adds to Git

In Lectures 6–8 you used GitHub as a place to store a repository and to host pull requests. GitHub offers much more around the repository. None of it changes how Git works; it is a layer of project-management and automation tools on top.

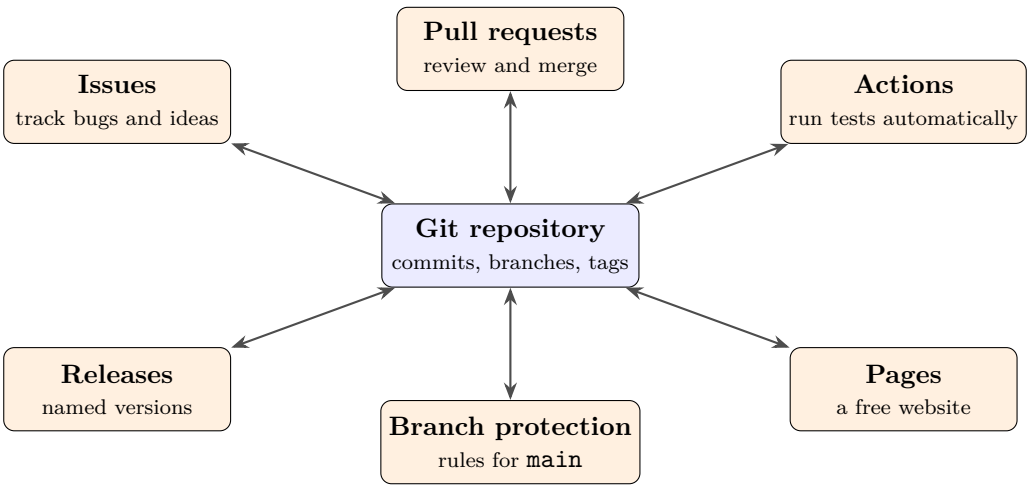


Figure 1: The Git repository at the centre, GitHub's features around it. This lecture covers all of the orange boxes except pull requests (Lecture 7).

2 Issues: tracking what needs doing

An **issue** is a numbered discussion about one thing: a bug, a question, an idea. Issues turn "someone should fix this" into a visible to-do list. Create one with **Issues** → **New issue**.

- A good **title** says what is wrong in one line: *Greeting prints "Helo" instead of "Hello".*
- A good **description** says how to see the problem, what you expected, and what happened.
- **Labels** classify (*bug*, *enhancement*, *good first issue*); **assignees** say who works on it; **milestones** group issues for a version.

Issues and pull requests share the numbering: `#12` anywhere in a comment or description becomes a link to issue or pull request number 12, and `@name` notifies a person. Descriptions are written in Markdown, so a task list works as a checklist:

```
- [x] reproduce the bug
- [ ] write a fix
- [ ] add a test
```

2.1 Closing issues automatically

When a pull request fixes an issue, you do not have to close the issue by hand. Put a **closing keyword** followed by the issue number in the pull request's description (or in a commit message):

```
Fix greeting typo

Closes #12
```

The keywords are *close(s/d)*, *fix(es/ed)* and *resolve(s/d)*, in any capitalisation. The issue is closed **when the change is merged into the default branch** (usually `main`), not when the pull request is opened. Before that, the issue page shows the pull request as linked.

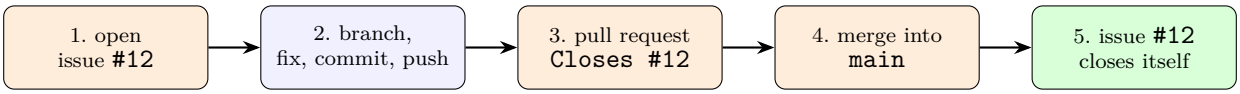


Figure 2: Issue to fix to closed issue. Orange steps happen on the website, blue steps are the Git commands you know.

2.2 Templates and the `.github` folder

GitHub looks in a folder called `.github` in your repository for helper files. Two are very useful for teamwork:

- `.github/ISSUE_TEMPLATE/bug_report.md`: pre-fills the form when someone opens an issue.
- `.github/pull_request_template.md`: pre-fills every pull request description.

A template for bug reports starts with a block of settings between lines of three dashes, called *front matter*, followed by Markdown:

```
---
name: Bug report
about: Something does not work
---

## What happened?

## What did you expect?
```

and a pull-request template is plain Markdown, for example:

```
## What does this change?

## Checklist
- [ ] The tests pass
- [ ] I read my own diff
```

These files are ordinary files: you commit and push them like any other, and they take effect on the default branch.

Quick fact

GitHub also offers *Projects*: a board with columns (to do, doing, done) whose cards are issues and pull requests. It is optional; many small teams get by with labels.

3 GitHub Actions: letting a robot run your tests

Continuous integration (CI) means: every time someone pushes, a machine automatically builds and tests the project, and reports the result next to the commit. Nobody forgets to run the tests, and a broken change is caught before it is merged. On GitHub this is done by **Actions**.

You describe what to run in a **workflow file**: a text file in the folder `.github/workflows/`, written in **YAML**. YAML is a simple format of `key: value` pairs where *indentation* (spaces, never tabs) shows nesting. Our project has a test script `test.sh`; this workflow runs it:

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run the tests
        run: bash test.sh
```

Line by line:

Line	Meaning
`name: CI`	The name shown in the Actions tab.
`on:`	*When* to run: on every push to `main`, and on every pull request.
`jobs:` / `test:`	A job called `test`. A workflow can have several; they run in parallel.
`runs-on: ubuntu-latest`	The kind of computer GitHub lends you: a fresh virtual machine with Linux.
`steps:`	The job's steps, run in order. A line starting with `-` begins a step.
`uses: actions/checkout@v4`	Use a ready-made *action*: it downloads your repository into the machine. (Without it the machine has no files.)
`run: bash test.sh`	Run a command, exactly as in your terminal. If it fails (non-zero exit), the job fails.

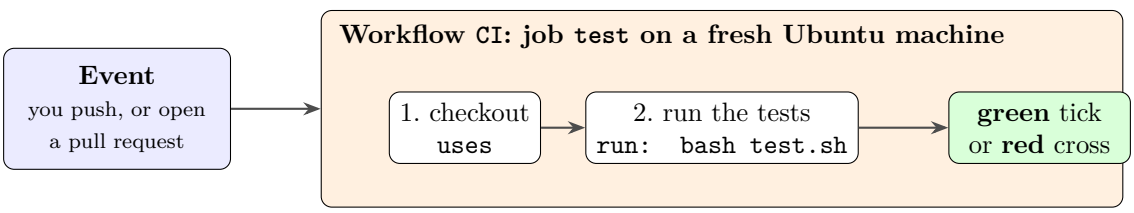


Figure 3: What happens on a push: the event starts the workflow, a fresh virtual machine runs the steps, and the result appears on the commit and the pull request.

Using it. Commit the file `.github/workflows/ci.yml` and push. Open the **Actions** tab of your repository: the run appears, yellow while it works, then a green tick or a red cross. Click it to read the log of every step. On a pull request the same result is shown as a *check* next to the merge button. If a step fails, read its log, fix the problem, and push again: the workflow runs again.

Pitfall

YAML is picky. Use spaces, never tabs. Keep the indentation exactly as shown: a step moved by one space changes its meaning. A broken file does not run at all; the Actions tab then shows an error with the line number.

Quick fact

Actions are free for public repositories; private repositories get a limited monthly allowance of free minutes. The version after `@` (`v4`) pins the action's major version; newer major versions appear over time, and GitHub's documentation lists the current one. Never write passwords into a workflow file; secrets go under Settings → Secrets and variables → Actions and are used as `${{ secrets.NAME }}`.

4 Branch protection: guarding `main`

In Lecture 8 you learned that rewriting a shared branch is dangerous, and in Lecture 7 that changes should be reviewed. **Branch protection** makes GitHub enforce this. In the repository's **Settings** (under *Branches*, or *Rules*) you can add a rule for `main`. The most useful options:

Option	Effect
Require a pull request before merging	Nobody can push straight to `main`; changes arrive through reviewed pull requests.
Require approvals	A pull request needs at least n approvals before the merge button works.
Require status checks to pass	The CI workflow must be green. A red cross blocks the merge.
Block force pushes	`git push --force` (even with a lease) is refused on `main`.
Restrict deletions	The branch cannot be deleted.

Together they give the team's central rule: *nothing reaches `main` unless it was reviewed and the tests pass.* (Branch protection is available for public repositories on every plan; for private repositories it depends on your plan.)

5 Tags and releases: naming a version

A branch label moves every time you commit. Sometimes you need a label that *stays put*: "this exact commit is version 1.0.0". That is a **tag**.

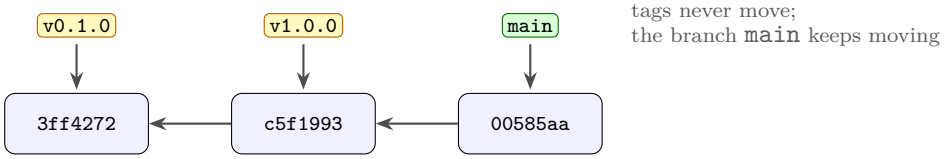


Figure 4: Tags mark specific commits. After more commits, `main` has moved on, but `v1.0.0` still names the same commit.

There are two kinds. A **lightweight** tag is just a name for a commit. An **annotated** tag is a small object of its own that also stores who made it, when, and a message; use annotated tags for releases.

```
$ git tag v0.1.0 3ff4272              # lightweight, on a given commit
$ git tag -a v1.0.0 -m "First release"    # annotated, on the current commit
$ git tag -n                          # list tags with their messages
v0.1.0          Add README
v1.0.0          First release
$ git show v1.0.0
tag v1.0.0
Tagger: Practice User <practice@example.com>

First release

commit <40-character ID>
...
```

Tags are not pushed automatically. Send them explicitly:

```
$ git push origin v0.1.0 v1.0.0        # or: git push origin --tags   (all tags)
 * [new tag]         v0.1.0 -> v0.1.0
 * [new tag]         v1.0.0 -> v1.0.0
```

Other tag commands: `git tag -d v1.0.0` deletes a local tag, `git push origin --delete v1.0.0` deletes it on the remote, and `git describe` tells you how far the current commit is from the nearest tag:

```
$ git describe --tags
v1.0.0-1-g00585aa           # one commit after v1.0.0
$ git switch --detach v0.1.0
HEAD is now at 3ff4272 Add README
```

Checking out a tag puts you on a detached `HEAD`, exactly as in Lecture 4: good for looking at the old version, not for committing.

5.1 Version numbers

Most projects use **semantic versioning**: `MAJOR.MINOR.PATCH`.

You made	Bump	From `1.0.0` to
a bug fix, nothing else changes	PATCH	`1.0.1`
a new feature that does not break anything	MINOR	`1.1.0`
a change that breaks how people use it	MAJOR	`2.0.0`

A **release** is GitHub's page for a tag. Under **Releases** → **Draft a new release** you choose a tag, write a title and notes (the button *Generate release notes* lists the merged pull requests for you), optionally attach files, and press *Publish release*. GitHub automatically adds downloadable ZIP and tar archives of the source code at that tag.

6 GitHub Pages and a healthy repository

GitHub Pages publishes files from your repository as a website, for free. In **Settings** → **Pages** choose to deploy from a branch (for example `main`, folder `/` or `/docs`). After a minute the site is at `https://YOURNAME.github.io/REPOSITORY/`. A single `index.html` (or `index.md`) is enough. It is a good place for project documentation or a personal page.

A repository that others can use usually contains:

- `README.md`: what it is and how to use it (shown on the front page);
- `LICENSE`: what others may do with the code (GitHub's *Add file* menu offers common licences; without one, the default is "all rights reserved");
- `.gitignore` (Lecture 2) and, for teamwork, `CONTRIBUTING.md` (how to propose changes) and the `.github` templates from above.

Pitfall

Security basics. Turn on two-factor authentication (Lecture 6). Never commit passwords, keys or tokens (Lecture 2): GitHub scans for known secrets, but history keeps them. GitHub's *Dependabot* can warn you about vulnerable libraries in a project; enable it under Settings → Code security when you have dependencies.

7 Homework 9

Homework 9: Issues, CI, and a release

Part A: practise with a "server" folder (no internet needed). The setup script creates `practice/hw9_server.git`, a repository that plays the role of GitHub, holding a tiny project whose test fails because of a typo. From the course folder:

```
$ bash codeHints/L09/setup_hw9.sh
$ cd practice
```

Do the tasks in order. To start over: `setup_hw9.sh --reset` from the course folder.

9.1 Clone

Clone the server into `hw9` and go in. Set your name and email (`git config user.name` ...) if they are not set globally.

9.2 See the failure

Run `bash test.sh`. It prints `FAIL`: this is what CI would show as a red cross.

9.3 Fix it

Correct the typo in `greet.sh` so that `bash test.sh` prints `PASS`.

9.4 Closing keyword

Commit the fix with a message that has the subject `Fix greeting typo` and a second paragraph `Closes #1`: `git commit -m "Fix greeting typo" -m "Closes #1"`.

9.5 The workflow

Create `.github/workflows/ci.yml` (use the example of this lecture: it checks out the code and runs `bash test.sh`, on pushes to `main` and on pull requests). Spaces only, no tabs. Commit it as `Add CI workflow`.

9.6 Templates

Create `.github/ISSUE_TEMPLATE/bug_report.md` (front matter with `name:` and `about:`, then

at least one `##` heading) and `.github/pull_request_template.md` (with a checklist line `- [ ] ...`). Commit them as `Add templates`.

9.7 Tags

Create the *lightweight* tag `v0.1.0` on the very first commit (`git rev-list --max-parents=0 HEAD` prints its ID) and the *annotated* tag `v1.0.0` (message `First release`) on your newest commit. Push everything: `git push origin main v0.1.0 v1.0.0`.

9.8 Questions

In `practice/hw9_answers.txt` (next to `hw9`; create it with `printf` or `echo`) write `next-bugfix=<version>` (the next version number after fixing another bug in `1.0.0`) and `issue-closes-when=<opened or merged>` (when does `Closes #12` close issue 12?).

Part B: the real thing on GitHub (optional, but this is the point). You need your GitHub account and SSH key (Lecture 6).

9.9 Publish and watch CI

On GitHub create a new *Public* repository `ci-practice` *without* a README. In `practice/hw9` run `git remote add github <your address>` and `git push github main v0.1.0 v1.0.0`. Open the **Actions** tab: your workflow ran. Add the line `repo=<you>/ci-practice` to `hw9_answers.txt`.

9.10 Issue to pull request to closed issue

Now (after the push) create Issue #1 (use the bug template; title: *Add a note to the README*). In `hw9` create a branch `readme-note`, add a line to `README.md`, commit, and `git push github readme-note`. Open a pull request into `main` with the description `Closes #1`; wait for the green check; merge it. Issue #1 should close by itself. Add `issue=1` to the answers file. (Create the issue only *after* the first push: the old commit `Closes #1` would otherwise close it at once.)

9.11 Release

On GitHub open **Releases** → **Draft a new release**, choose the tag `v1.0.0`, press *Generate release notes*, and publish.

9.12 Explore (not checked)

Under Settings, look at branch protection and GitHub Pages. In Settings you can also protect `main` so that CI must pass.

Check yourself: from the course folder run `bash codeHints/L09/check_hw9.sh`. Part A needs no network. Part B is checked if `hw9_answers.txt` contains `repo=`; the checker reads your public repository through GitHub's public interface (your issue, release and latest Actions run) and says so if it cannot reach GitHub. Then compare with the solutions at the end of this lecture.

8 Summary

- Issues track work; `Closes #12` in a pull request closes the issue when the change reaches `main`. Templates live in `.github/`.
- Actions run a YAML workflow from `.github/workflows/` on each push or pull request: `on`, `jobs`, `runs-on`, `steps`, `uses`, `run`. Green tick or red cross.
- Branch protection (pull request, approvals, passing checks, no force push) enforces the team's rules on `main`.
- Tags name fixed commits (annotated for releases; push them explicitly); releases are GitHub pages for tags; version as `MAJOR.MINOR.PATCH`. Pages publishes a free website. (Further reading: GitHub Docs, "About issues", "Understanding GitHub Actions"; *Pro Git*, Chapter 2.6.)

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 9

Part A.

```
$ bash codeHints/L09/setup_hw9.sh
$ cd practice
$ git clone hw9_server.git hw9
$ cd hw9
$ bash test.sh
FAIL: got 'Helo, world'
$ sed -i.bak 's/Helo/Hello/' greet.sh && rm greet.sh.bak    # or edit by hand
$ bash test.sh
PASS
$ git commit -am "Fix greeting typo" -m "Closes #1"
$ mkdir -p .github/workflows .github/ISSUE_TEMPLATE
```

Create `.github/workflows/ci.yml` (spaces only):

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run the tests
        run: bash test.sh
```

Create `.github/ISSUE_TEMPLATE/bug_report.md` (the front matter with `name:` and `about:` and the two headings of the lecture) and `.github/pull_request_template.md` (the heading and the checklist of the lecture). Then:

```
$ git add .github
$ git commit -m "Add CI workflow and templates"
$ git tag v0.1.0 $(git rev-list --max-parents=0 HEAD)
$ git tag -a v1.0.0 -m "First release"
$ git push origin main v0.1.0 v1.0.0
$ printf 'next-bugfix=1.0.1\nissue-closes-when=merged\n' > ../hw9_answers.txt
```

(Committing the workflow and the templates in two commits, as the task text says, works equally well.)

9.3–9.4. The checker runs your project's `test.sh` on a copy of the server's `main`, which is exactly what the workflow does on GitHub. `git commit -m ... -m ...` makes the second `-m` a separate paragraph of the message; GitHub looks for the keyword and the issue number anywhere in it.

9.5. Without `uses: actions/checkout` the virtual machine would have no files, so `bash test.sh` would fail. The workflow file must be in `.github/workflows` (with an *s*).

9.7. The lightweight tag is a bare name for the first commit; the annotated tag has its own object with your message, so `git show v1.0.0` shows a *Tagger*. Tags are only pushed when you name them (or use `--tags`).

9.8. Fixing a bug in `1.0.0` gives the patch version `1.0.1`. `Closes #12` closes the issue when the change is *merged* into the default branch, not when the pull request is opened.

Part B.

```
(GitHub) New repository: ci-practice, Public, no README
$ git remote add github git@github.com:YOURNAME/ci-practice.git
$ git push github main v0.1.0 v1.0.0
(GitHub) Actions tab: the "CI" workflow ran: green tick
$ printf 'repo=YOURNAME/ci-practice\n' >> ../hw9_answers.txt
(GitHub) Issues > New issue "Add a note to the README"  -> number 1
$ git switch -c readme-note
$ echo "Notes coming soon." >> README.md
$ git commit -am "Add a note to the README"
$ git push github readme-note
(GitHub) Open the pull request, description "Closes #1"; wait for the green check;
    Merge
(GitHub) Issue #1 is now closed
$ printf 'issue=1\n' >> ../hw9_answers.txt
(GitHub) Releases > Draft a new release > tag v1.0.0 > Generate release notes > Publish
```

Replace `YOURNAME` with your GitHub user name. The checker needs the answers file to contain `repo=`, `issue=`, and a remote of `practice/hw9` that points at that repository. It then asks GitHub's public interface whether the issue is closed, whether a release exists for `v1.0.0`, and whether the latest Actions run on `main` succeeded. If the last check is red, open the run's log in the Actions tab.

Common mistakes.

- A *tab* character in the workflow (the checker reports it) or a folder called `.github/workflow` instead of `workflows`.
- Creating Issue #1 *before* the first push: the old commit message `Closes #1` then closes it at once, and the pull request closes nothing. The check passes anyway, but you missed the exercise. Push first, then create the issue.
- Forgetting to push the tags: `git push origin main` sends no tags.
- Creating the GitHub repository with a README: the first push is rejected because the histories differ. Create it empty.
- A private repository: GitHub's public interface cannot see it, so the checker reports that it cannot find it.
- Writing `issue-closes-when=opened`: the issue closes when the change is *merged*.